

HDLBits - Circuits / Sequential Logic / Finite State Machines 3



yfwang

Follow

7 min read · Jun 18, 2023



1

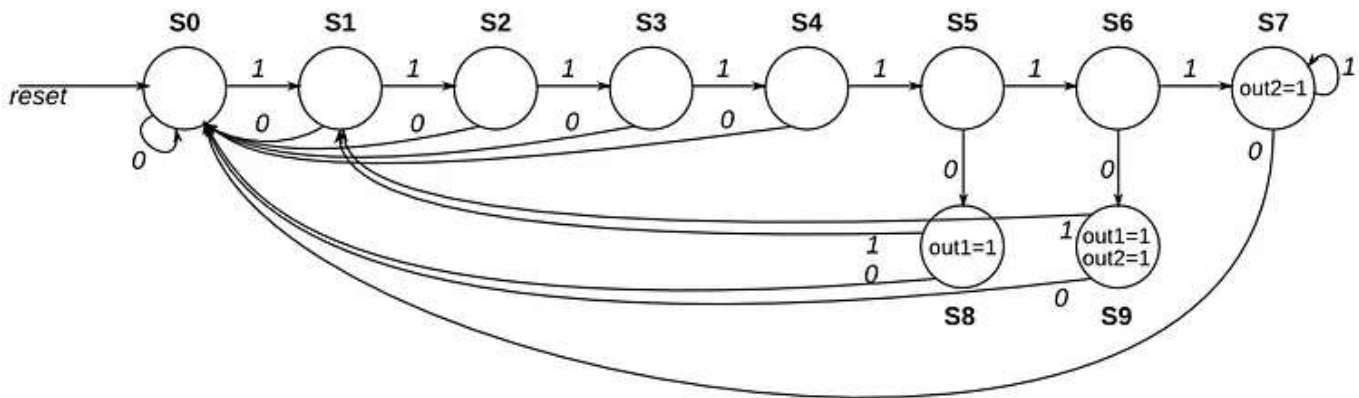


Outline

- One-hot FSM
- PS/2 packet parser
- PS/2 packet parser and datapath
- Serial receiver
- Serial receiver and datapath
- Serial receiver with parity checking
- Sequence recognition

One-hot FSM

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_onehot



Write the state machine in one-hot code according to the status diagram of the question.

```

module top_module(
    input in,
    input [9:0] state,
    output [9:0] next_state,
    output out1,
    output out2);

parameter S0=4'd0, S1=4'd1, S2=4'd2, S3=4'd3, S4=4'd4, S5=4'd5, S6=4'd6, S7=4'd7, S8=4'd8, S9=4'd9;

//next state
assign next_state[S0] = ~in&(state[S0]|state[S1]|state[S2]|state[S3]|state[S4]|state[S5]|state[S6]|state[S7]);
assign next_state[S1] = (in&state[S0])|(in&state[S8])|(in&state[S9]);
assign next_state[S2] = in&state[S1];
assign next_state[S3] = in&state[S2];
assign next_state[S4] = in&state[S3];
assign next_state[S5] = in&state[S4];
assign next_state[S6] = in&state[S5];
assign next_state[S7] = (in&state[S6])|(in&state[S7]);
assign next_state[S8] = ~in&state[S5];
assign next_state[S9] = ~in&state[S6];

//output
assign out1 = state[S8]|state[S9];
assign out2 = state[S7];
  
```

```

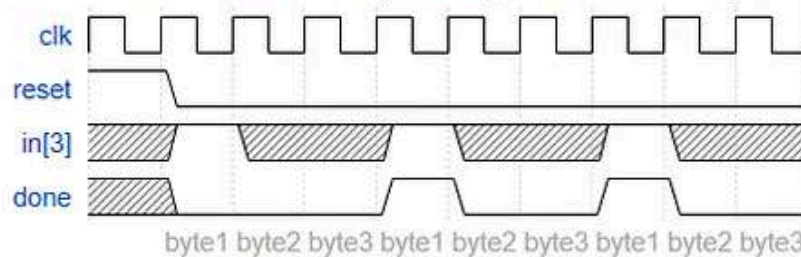
assign out2 = state[S7] | state[S9];
endmodule

```

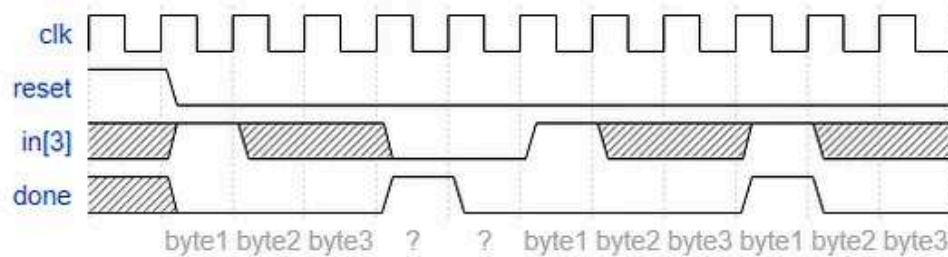
PS/2 packet parser

- 題目連結 : https://hdlbits.01xz.net/wiki/Fsm_ps2

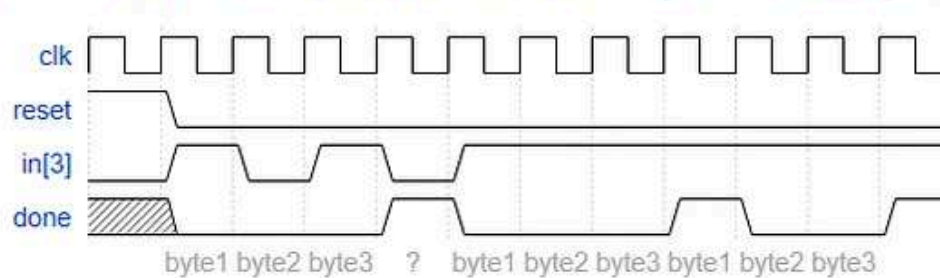
Under error-free conditions, every three bytes form a message:



When an error occurs, search for byte 1:



Note that this is not the same as a 1xx sequence recognizer. Overlapping sequences are not allowed here:



實作 PS/2 傳輸介面。算是一種序列檢測，當 $in[3] == 1$ 的時候開始序列，每經過 3 個 byte 輸出 **done**，但輸出 **done** 時必須 $in[3] == 1$ ，若 $in[3] != 1$ 則

丟棄 byte 直到下一次 $in[3] == 1$ 發生。

Get yfwang's stories in your inbox

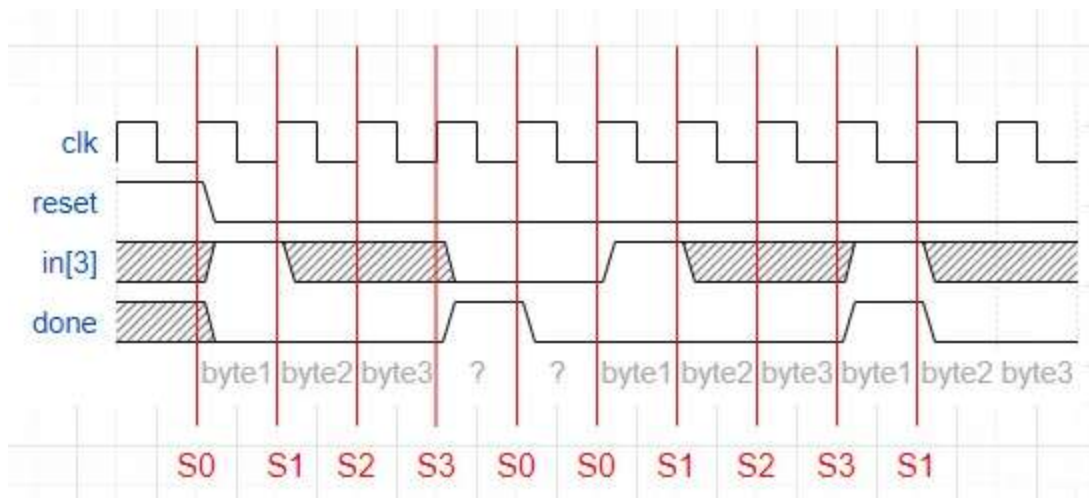
Join Medium for free to get updates from this writer.

Enter your email

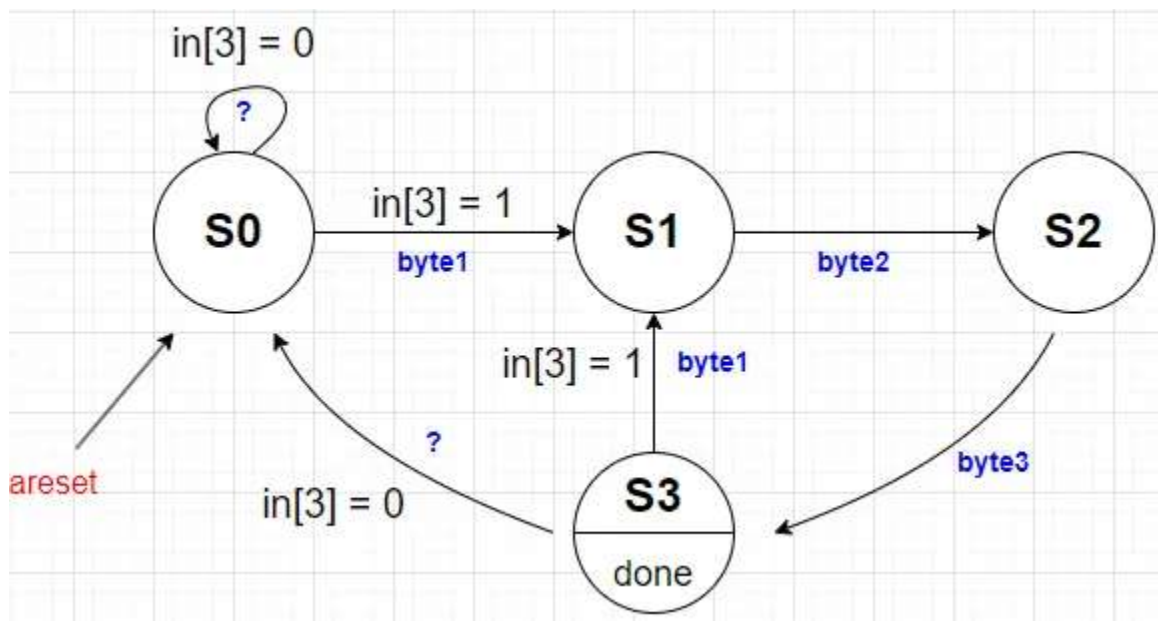
Subscribe

☐ Remember me for faster sign in

這裡指的丟棄 byte 就是上述時序圖的 ? 部分，可以發現下一次的 **byte1** 是從 $in[3] == 1$ 後開始。



設計狀態圖時要看的是 **clk posedge** 的時候，這裡我假設 4 個狀態 $S_0 \sim S_3$ ，其中 S_0 是 **reset** 起始點，而 S_3 代表 **done** 要拉 H，若 **done** 拉 H 時 $in[3] == 1$ 則作為 **byte1** 往下進行序列，狀態圖如下：



```

module top_module(
    input clk,
    input [7:0] in,
    input reset,    // Synchronous reset
    output done); //

parameter S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
reg [2:0] state, next_state;

// State transition logic (combinational)
always @(*) begin
    case (state)
        S0: next_state = in[3] ? S1:S0;
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = in[3] ? S1:S0;
        default: next_state = S0;
    endcase
end

// State flip-flops (sequential)
always @(posedge clk) begin
    state <= (reset) ? S0:next_state ;
end

// Output logic
always @(posedge clk) begin
    if (reset) begin
        done <= 0;
    end
end
  
```

```

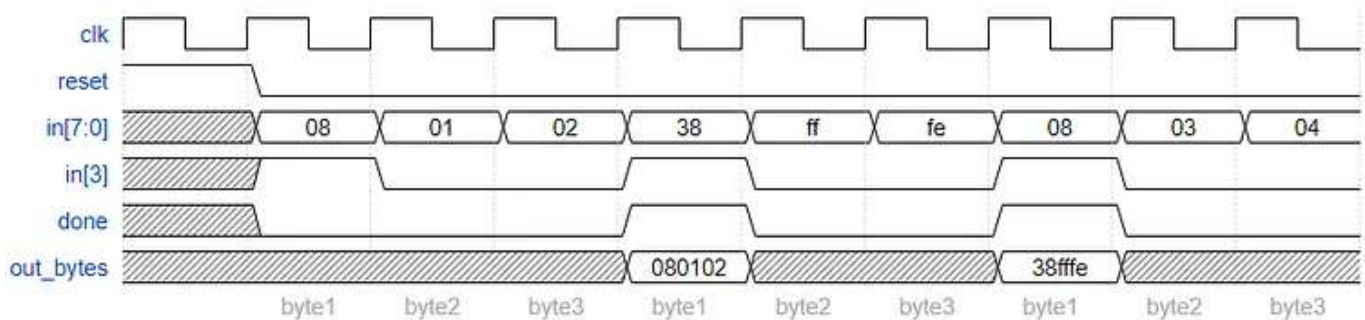
end
else begin
    case (next_state)
        S0: done<=0;
        S1: done<=0;
        S2: take <=0;
        S3: take <=1;
        default: Take <=0;
    endcase
end
end

endmodule

```

PS/2 packet parser and datapath

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_ps2data



Submit the above question and collect more data. **done** OutputTime

out_bytes Are the first three groups in composition, so just plug data in from behind, in the timing diagram ? The data part will be flush.

```

module top_module(
    input clk,

```

```
input [7:0] in,
input reset,    // Synchronous reset
output [23:0] out_bytes,
output done); //

// FSM from fsm_ps2
parameter S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
reg [2:0] state, next_state;

// State transition logic (combinational)
always @(*) begin
    case (state)
        S0: next_state = in[3] ? S1:S0;
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = in[3] ? S1:S0;
        default: next_state = S0;
    endcase
end

// State flip-flops (sequential)
always @(posedge clk) begin
    state <= (reset) ? S0:next_state ;
end

// Output logic
always @(posedge clk) begin
    if (reset) begin
        done <= 0;
    end
    else begin
        case (next_state)
            S0: done<=0;
            S1: done<=0;
            S2: done<=0;
            S3: done<=1;
            default: done<=0;
        endcase
        out_bytes<={out_bytes[15:0],in[7:0]};
    end
end

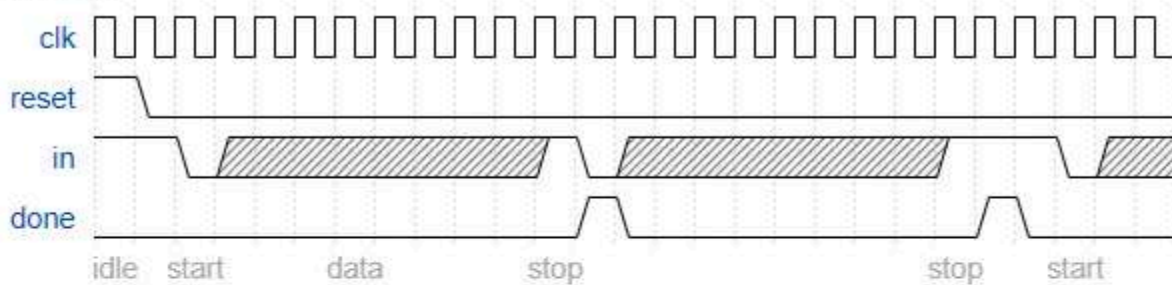
// New: Datapath to store incoming bytes.

endmodule
```

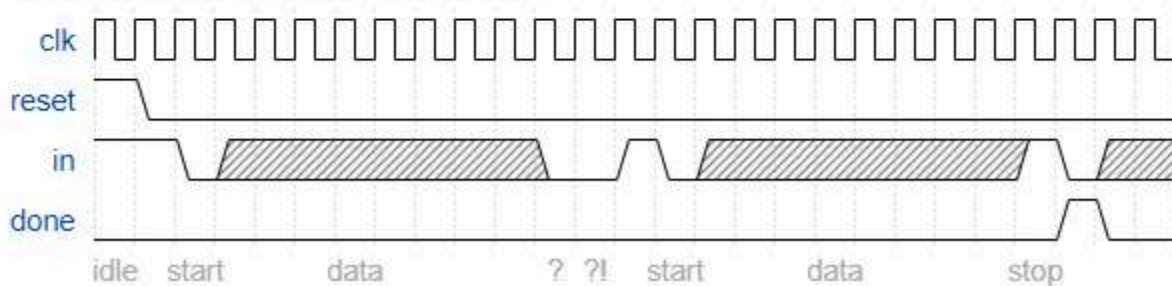
Serial receiver

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_serial

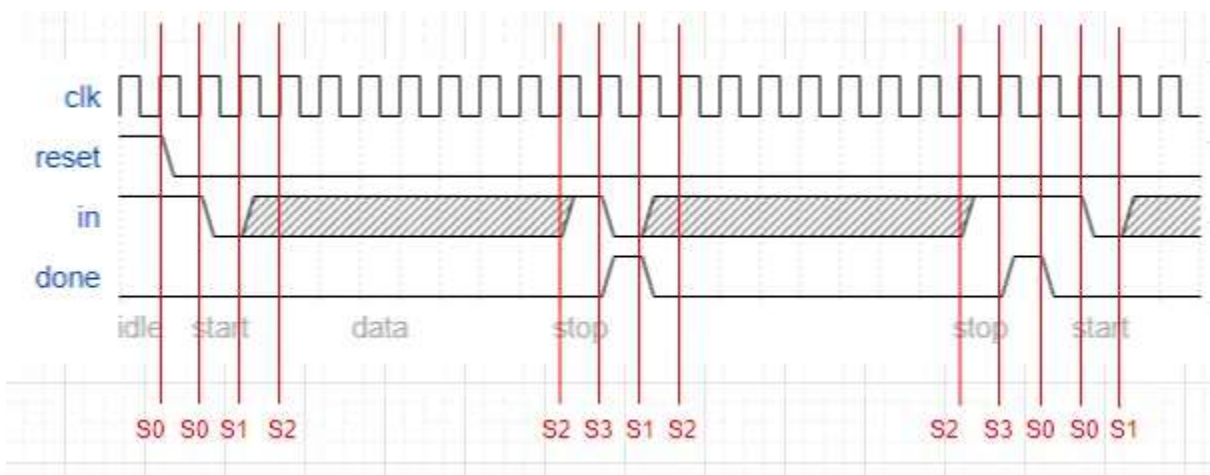
Error-free:



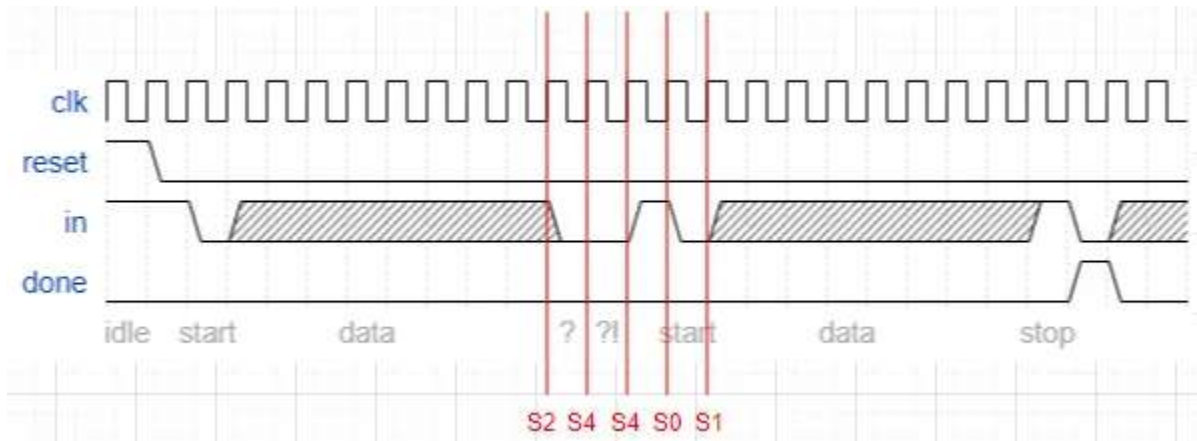
Stop bit not found. First byte is discarded:



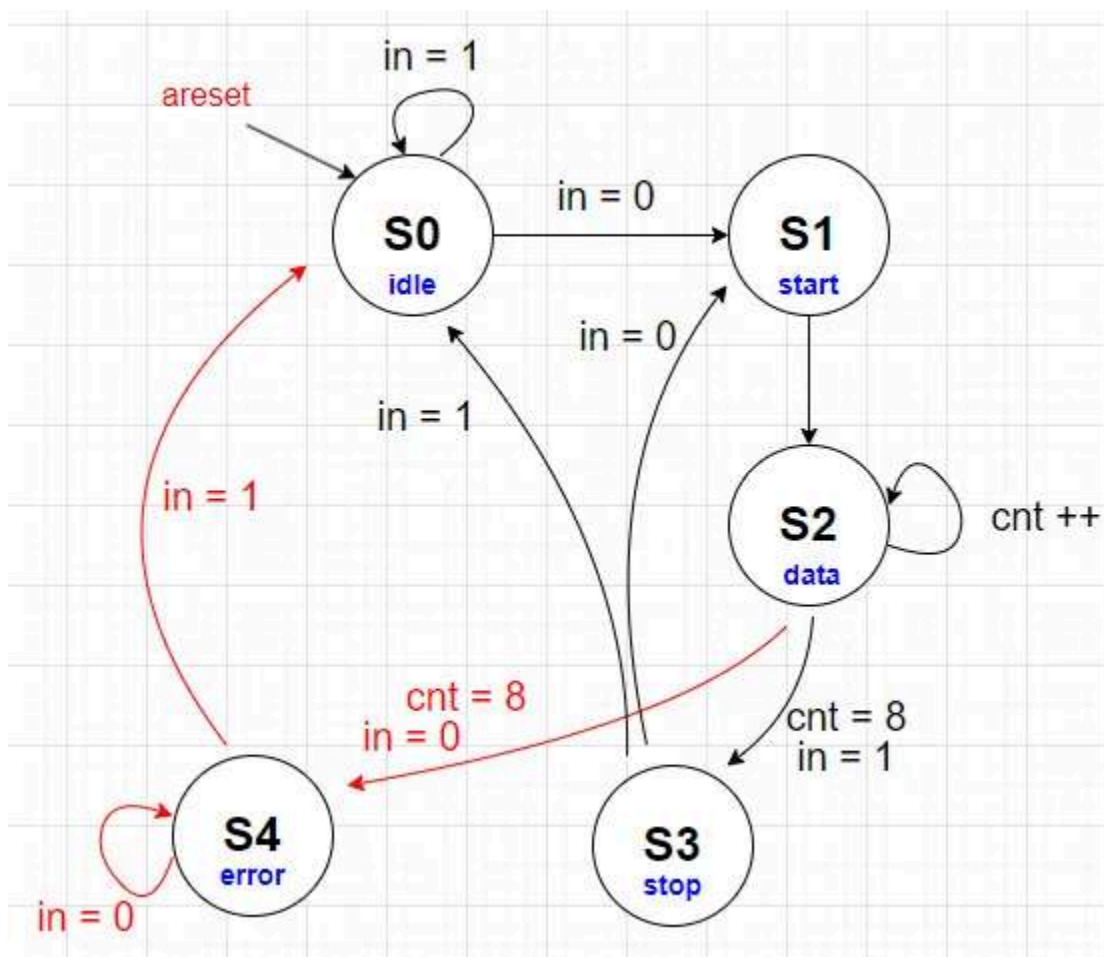
Implement serial. According to the timing diagram, when in is **L**.
Receive **8 bits** Data, and in is **H**. is regarded as a stop position.



And there is the same discard mechanism as PS/2, as assumed in the following state diagram **error** Status s_4 .



According to two figures **clk posedge** (vertical red line) Draw a status diagram.



```
module top_module(  
    input clk,  
    input in,  
    input reset,    // Synchronous reset  
    output done  
);  
  
parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100;  
reg [2:0] state, next_state;  
reg [3:0] cnt;  
  
// State transition logic (combinational)  
always @(*) begin  
    case (state)  
        S0: next_state = in ? S0:S1; //idle or start  
        S1: next_state = S2; //data  
        S2: next_state = (cnt==8) ? (in?S3:S4):S2; //cnt==8->S3orS4  
        S3: next_state = in ? S0:S1; //idle or start  
        S4: next_state = in ? S0:S4; //idle or keep  
        default: next_state = S0;  
    endcase  
end  
  
// State flip-flops (sequential)  
always @(posedge clk) begin  
    state <= (reset) ? S0:next_state ;  
end  
  
// Output logic  
always @(posedge clk) begin  
    if (reset) begin  
        done <= 0;  
    end  
    else begin  
        case (next_state)  
            S3: done<=1;  
            default: done<=0;  
        endcase  
    end  
end  
  
always @(posedge clk) begin  
    if (reset) begin  
        cnt <= 0;  
    end  
    else begin  
        case (next_state)  
            S1: cnt<=0;  
        endcase  
    end  
end
```

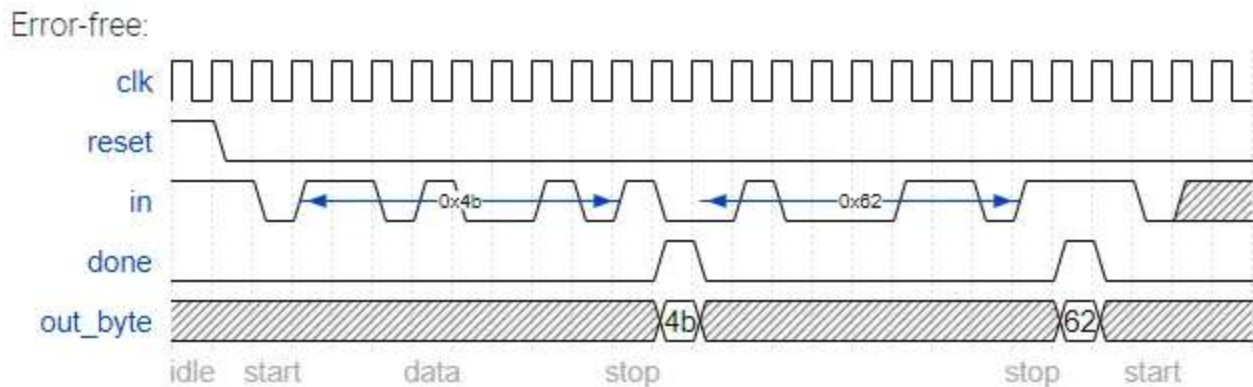
```

        S2: cnt<=cnt+1;
        default: cnt<=cnt;
    endcase
end
end
endmodule

```

Serial receiver and datapath

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_serialdata



Submit the above question and collect more data. Create a reg here `data` To store data, when the status arrives `s3` It is output only when, please note that the data transmission here is **LowestValid** bit **LSB** , the earliest to go in ranks the lowest.

```

modules top_module(
    input clk,
    input in,

```

```

input reset,    // Synchronous reset
output [7:0] out_byte,
output done
); //

// Use FSM from Fsm_serial
parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100;
reg [2:0] state, next_state;
reg [3:0] cnt;
reg [7:0] data;

// State transition logic (combinational)
always @(*) begin
    case (state)
        S0: next_state = in ? S0:S1; //idle or start
        S1: next_state = S2; //data
        S2: next_state = (cnt==8) ? (in?S3:S4):S2; //cnt==8->S3orS4
        S3: next_state = in ? S0:S1; //idle or start
        S4: next_state = in ? S0:S4; //idle or keep
        default: next_state = S0;
    endcase
end

// State flip-flops (sequential)
always @(posedge clk) begin
    state <= (reset) ? S0:next_state ;
end

// Output logic
always @(posedge clk) begin
    if (reset) begin
        done <= 0;
    end
    else begin
        case (next_state)
            S3: done<=1;
            default: done<=0;
        endcase
    end
end

always @(posedge clk) begin
    if (reset) begin
        cnt <= 0;
    end
    else begin
        case (next_state)
            S1: cnt<=0;
            S2: cnt<=cnt+1;
            default: cnt<=cnt;
        endcase
    end
end

```

```
        endcase
    end
end

always @(posedge clk) begin
    if (reset) begin
        data <= 0;
    end
    else begin
        case (next_state)
            S1: data<=0;
            S2: data<={in,data[7:1]};
            default: data<=data;
        endcase
    end
end

always @(posedge clk) begin
    if (reset) begin
        out_byte <= 0;
    end
    else begin
        case (next_state)
            S3: out_byte <= data;
            default: out_byte <= 0;
        endcase
    end
end

//assign out_byte = data;

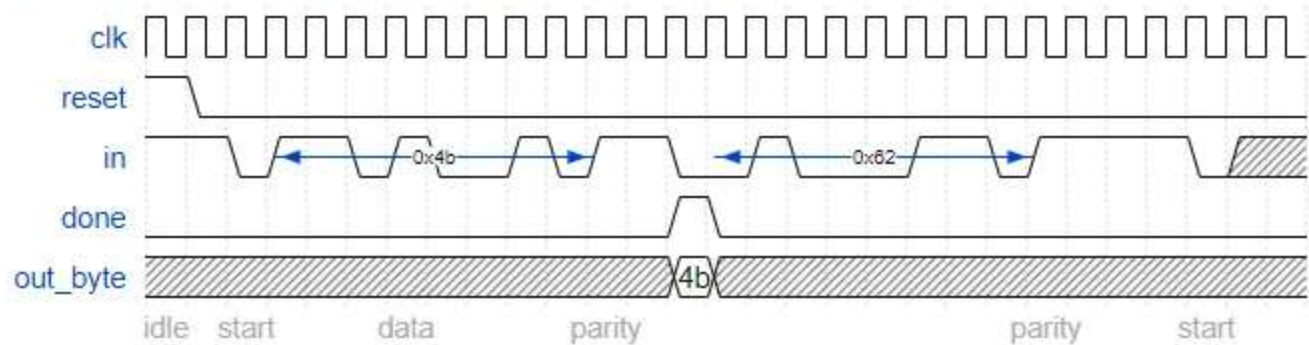
// New: Datapath to latch input bits.

endmodule
```

Serial receiver with parity checking

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_serialdp

No framing errors. Odd parity passes for first byte, fails for second byte.



Two questions are presented, with new additions **Parity**. The question already provides modules for parity:

```
module parity (
    input clk,
    input reset,
    input in,
    output reg odd);

    always @(posedge clk)
        if (reset) odd <= 0;
        else if (in) odd <= ~odd;

endmodule
```

So the code needs to be added **Input rst** And **Output odd** , used to reset odd Calculation & Output odd For sequence judgment done Output value.

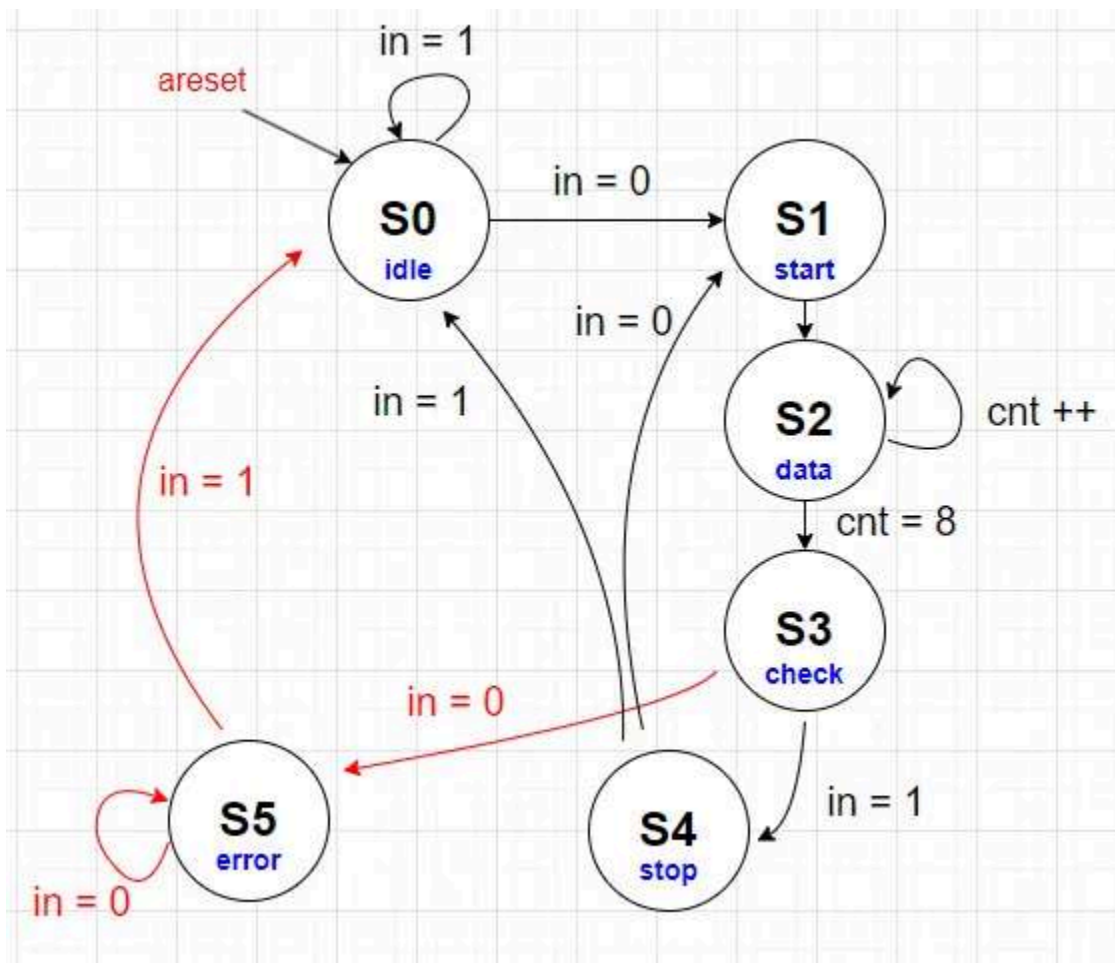
```
wire odd, rst;

parity parity_inst(
    .clk(clk),
    .reset(rst),
    .in(in),
```

```
.odd(odd)  
);
```

And add a **checkstatus**, and check the results odd Send done 。

```
always @(posedge clk) begin  
    if (reset) begin  
        done <= 0;  
    end  
    else begin  
        case (next_state)  
            S4: done <= (odd);  
            default: done <= 0;  
        endcase  
    end  
end
```



```

modules top_module(
    input clk,
    input in,
    input reset,    // Synchronous reset
    output [7:0] out_byte,
    output done
); //

// Use FSM from Fsm_serial
parameter S0=3'b000, S1=3'b001, S2=3'b010, S3=3'b011, S4=3'b100, S5=3'b101;
reg [2:0] state, next_state;
reg [3:0] cnt;
reg [7:0] data;
wire odd, rst;

// State transition logic (combinational)
always @(*) begin
    case (state)
        S0: next_state = in ? S0:S1; //idle or start
        S1: next_state = S2; //data
        S2: next_state = (cnt==8) ? S3:S2; //cnt==8->check
    endcase
end
  
```



```
S3: next_state = in ? S4:S5; //stop or error
S4: next_state = in ? S0:S1; //idle or start
S5: next_state = in ? S0:S5; //idle or keep
    default: next_state = S0;
endcase
end

// State flip-flops (sequential)
always @(posedge clk) begin
    state <= (reset) ? S0:next_state ;
end

// Output logic
always @(posedge clk) begin
    if (reset) begin
        cnt <= 0;
    end
    else begin
        case (next_state)
            S1: cnt<=0;
            S2: cnt<=cnt+1;
            default: cnt<=cnt;
        endcase
    end
end

always @(posedge clk) begin
    if (reset) begin
        data <= 0;
    end
    else begin
        case (next_state)
            S1: data<=0;
            S2: data<={in,data[7:1]};
            default: data<=data;
        endcase
    end
end

assign rst = (reset) | (next_state==S1);
parity parity_inst(
    .clk(clk),
    .reset(rst),
    .in(in),
    .odd(odd)
);

always @(posedge clk) begin
    if (reset) begin
        done <= 0;
    end
end
```

```
end
else begin
    case (next_state)
        S4: done <= (odd);
        default: done <= 0;
    endcase
end
end
//assign done = (state == S4) && (~odd); //odd reserve

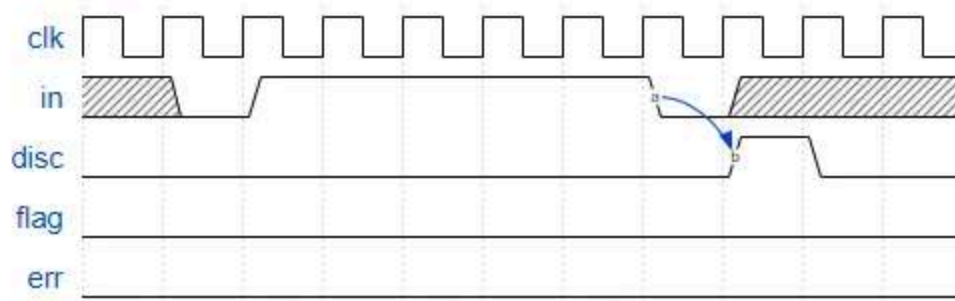
always @(posedge clk) begin
    if (reset) begin
        out_byte <= 0;
    end
    else begin
        case (next_state)
            S4: out_byte <= data;
            default: out_byte <= 0;
        endcase
    end
end
//assign out_byte = data;

endmodule
```

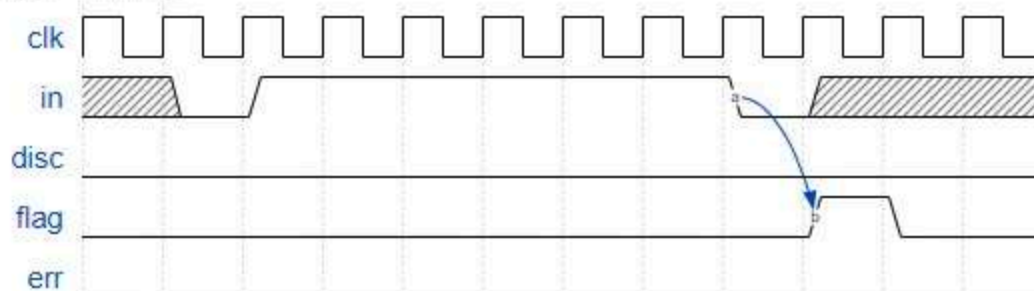
Sequence recognition

- 題目連結: https://hdlbits.01xz.net/wiki/Fsm_hdlc

Discard 0111110:



Flag 01111110:



Open in app ↗

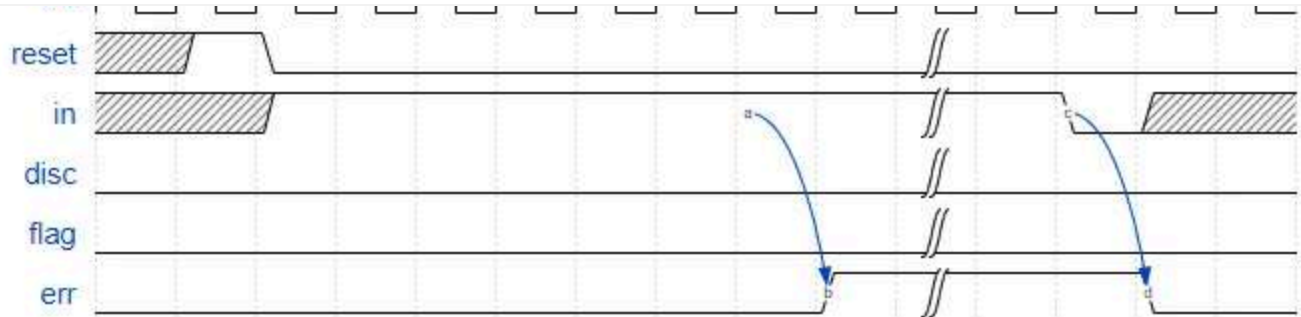
Sign up

Sign in

Medium

Search

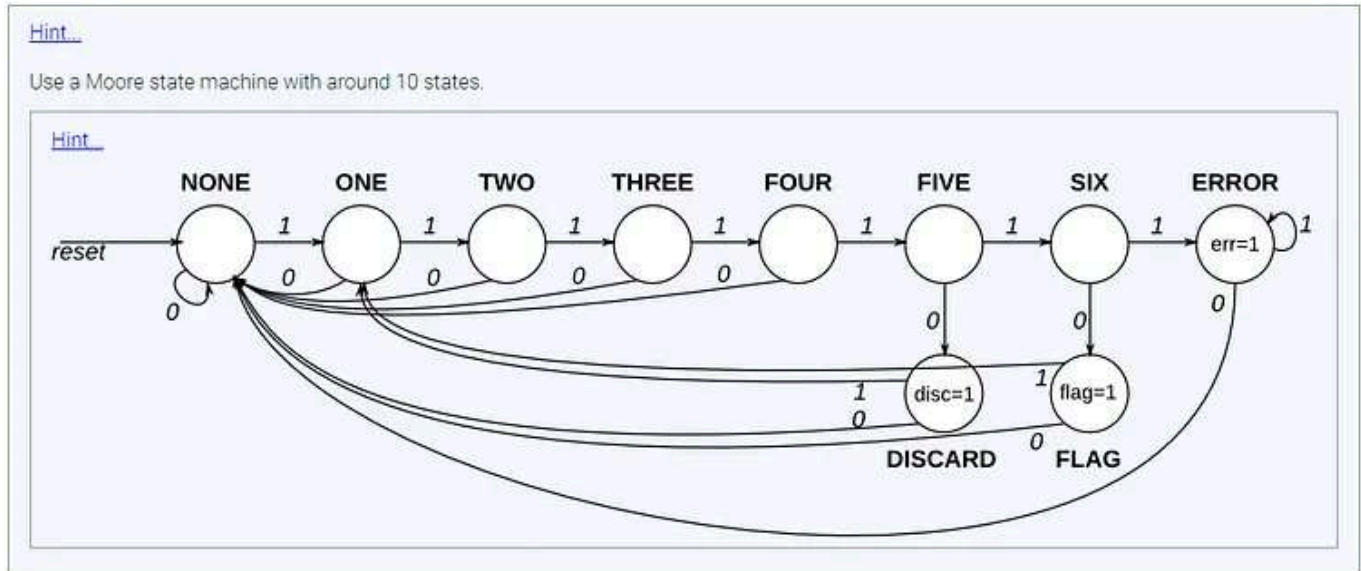
Write



Calculation in this question $in == 1$ **Continuoustimes**, output if 5 times in a row $disc$, 6 outputs $flag$, more than 7 times is err .

- 0111110: Signal a bit needs to be discarded ($disc$).
- 01111110: Flag the beginning/end of a frame ($flag$).
- 01111111...: Error (7 or more 1s) (err).

The figure below is the status diagram provided by the title Hint. Just write the code based on the figure.



By the way, I just discovered that the Hint of the question has a status diagram. . .

```

module top_module(
    input clk,
    input reset,    // Synchronous reset
    input in,
    output disc,
    output flag,
    output err);

parameter IDLE=4'd0, ONE=4'd1, TWO=4'd2, THREE=4'd3, FOUR=4'd4, FIVE=4'd5, SIX=4'd6, ERROR=4'd7, DISC=4'd8, FLAG=4'd9;
reg [3:0] state ,next_state;

// State transition logic (combinational)
always @(*)begin
    case(state)
        IDLE: next_state = in? ONE:IDLE;
        ONE: next_state = in? TWO:IDLE;
        TWO: next_state = in? THREE:IDLE;
        THREE:next_state = in? FOUR:IDLE;
        FOUR: next_state = in? FIVE:IDLE;
        FIVE: next_state = in? SIX:DISC;
        SIX: next_state = in? ERROR:FLAG;
        ERROR: next_state = in? ERROR:IDLE;
        DISC: next_state = in? IDLE:FLAG;
        FLAG: next_state = in? ERROR:IDLE;
    endcase
end
  
```

```
SIX:  next_state = in? ERR:FLAG;
ERR:  next_state = in? ERR:IDLE;
DISC: next_state = in? ONE:IDLE;
FLAG: next_state = in? ONE:IDLE;
default: next_state = IDLE;
endcase
end

// State flip-flops (sequential)
always @(posedge clk) begin
    state <= (reset) ? IDLE:next_state ;
end

// Output logic
assign disc = (state == DISC);
assign flag = (state == FLAG);
assign err  = (state == ERR);

endmodule
```

Chinese

Verilog

**Written by yfwang**

34 followers · 1 following

Follow

**No responses yet**

Write a response